

CHAPTER 4

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It bridges the gap between requirements analysis and system implementation.

4.1 BASIC MODULES

Basic modules are the main functional sections into which the system is divided so that it is easier to develop, understand, and maintain.

MODULES

- User Management Module
- Car Management Module
- Booking & Reservation Module
- Payment Processing Module
- Report & Analytics Module
- Notification Module
- Authentication Module
- Database Module

User Management Module: Handles all operations related to users, such as customers, car owners, and administrators. It manages user registration, profile updates, access control, and role-based permissions to ensure that each type of user interacts with the system according to their privileges.

Car Management Module: Allows car owners (or admins) to add, update, and manage car listings. It handles details such as brand, model, registration number, seating capacity, availability status, rental price per day, and associated images.

Booking & Reservation Module: Responsible for processing rental bookings by customers. It manages searching cars, selecting rental periods, checking availability, calculating costs, confirming bookings, and storing reservation details.

Payment Processing Module: Manages secure payments for bookings. It validates payment details, processes transactions via different gateways (credit/debit card, UPI, wallets), and updates booking status based on payment confirmation.

Report & Analytics Module: Generates and exports reports related to car availability, revenue, booking trends, and customer activity. It provides insights to car owners and admins to monitor fleet performance and financial statistics.

Notification Module: Sends alerts and updates to users. Customers receive booking confirmations, owners receive notifications on new bookings, and admins receive alerts on important system activities. Notifications are sent via email or in-app messages.

Authentication Module: Ensures that only authorized users can access the system. It manages login, registration, password encryption, and secure session handling to protect sensitive user and payment information.

Database Module: Stores, retrieves, updates, and manages all system data. It serves as the central hub for user details, car information, booking records, payment transactions, and reports. It ensures data integrity, security, and fast retrieval for smooth operations.

4.2 DATA DESIGN

Data design in this project structures and organizes the information related to users, cars, bookings, and payments so that it can be stored, retrieved, and updated efficiently. It ensures consistent and reliable information flow across all modules.

- Ensure completeness: All required data such as user details, car information, booking records, and payment details are properly captured.
- Improve efficiency: Data storage is optimized to reduce redundancy and enable fast access.

- Maintain consistency: Duplicate/conflicting records are minimized.
- Support integration: Smooth communication between modules like User Management, Car Management, and Booking & Payment Processing.

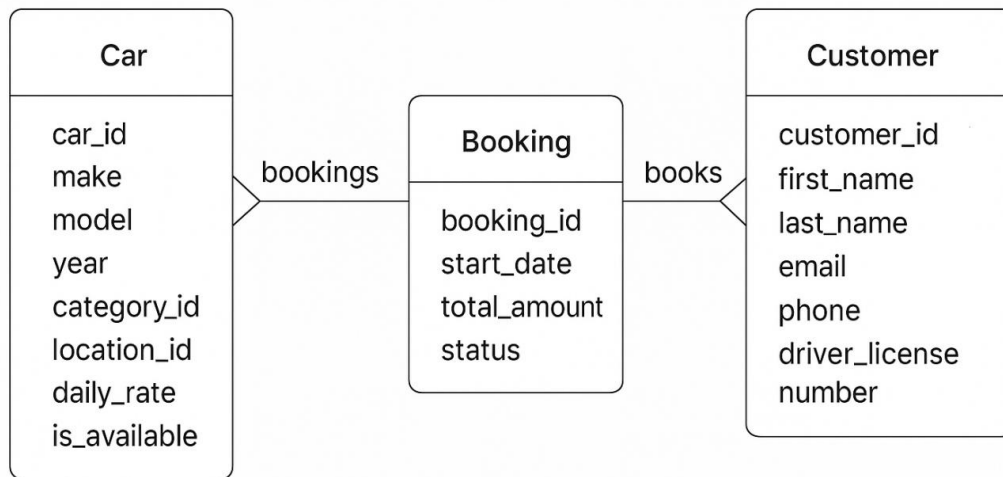


Figure 4.1 Schema design

4.3 PROCEDURAL DESIGN

Procedural design defines the sequence of operations, workflows, and how different system components interact to achieve functionality. It helps in structuring the system into smaller, manageable processes.

4.3.1 LOGICAL DIAGRAM

A logical diagram shows how parts of the system are logically connected, focusing on flow of information and processes.

- Users (Customers, Car Owners, Admins) interact via the User Interface (Frontend).
- Application Layer (Backend) processes booking, payments, car management, authentication, and reporting.
- Database Layer (PostgreSQL/Supabase) stores all user, car, booking, and payment data.

- Payment Gateway ensures secure rental payment transactions.

USE CASE DIAGRAM

A Use Case Diagram visually represents how different actors (Customer, Car Owner, Admin) interact with the Self-Drive Car Booking System to achieve their goals.

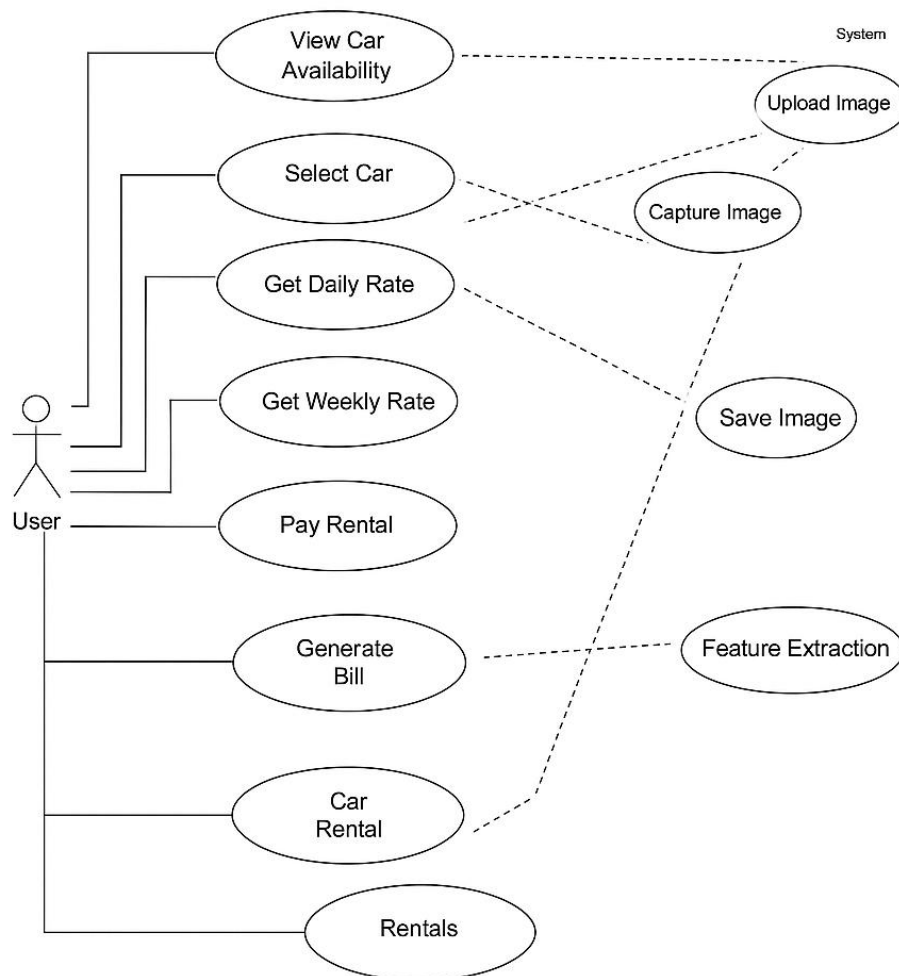


Figure 4.2 Use Case Diagram

4.4 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows through a system. It shows the sources of data, the processes that transform it, the data stores where it is kept, and the outputs that result from the system.

DFDs provide a clear visualization of how inputs are processed to generate outputs, making it easier to analyze and improve the system's design.

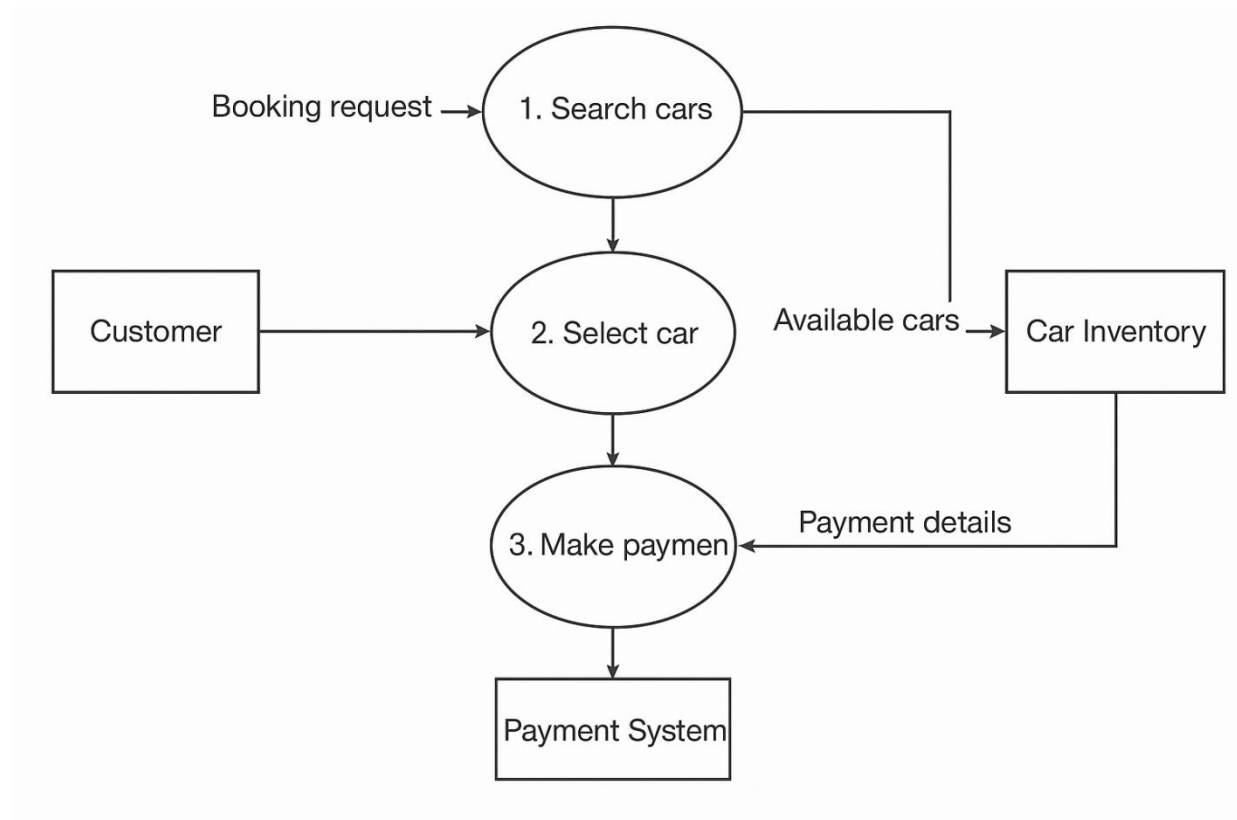


Figure 4.3 Data Flow Diagram

DATA FLOW COMPONENTS

A Data Flow Diagram (DFD) is a visual representation that shows how data moves through a system. It illustrates where data comes from (input), how it is processed (processes), where it is stored (data stores), and where it goes (output). It helps developers, analysts, and stakeholders understand the system's workflow clearly before implementation.

External Entities – Sources or destinations of data outside the system.

- Shape – Rectangle

Processes – Activities that transform input data into output data.

- Shape – Circle or Round Rectangle

Data Stores – Places where data is stored for future use.

- Shape - Open-ended Rectangle (two parallel lines or one side open)

Data Flows – Paths that show the movement of data between entities, processes, and data stores.

- Shape – Arrow Lines

4.5 CLASS DIAGRAM

A Class Diagram is a type of UML (Unified Modeling Language) diagram that represents the static structure of a system. It shows the system's classes, their attributes (data), methods (behavior), and the relationships between them. Class diagrams are essential in object-oriented design because they provide a blueprint for how different objects will interact within the system.

- Defining the data structure of the system.
- Specifying the responsibilities of each class.
- Showing how classes are related through association, inheritance, aggregation, or composition.
- Improving clarity for both developers and stakeholders before actual coding begins.

Components of a Class Diagram

1. Classes: User, Car, Booking, Payment, Review.
2. Attributes: Each class has properties like User → (userId, name, email, role).
3. Methods: Example – User → register(), login(); Car → addCar(), updateCar(); Booking → confirmBooking().
4. Relationships:
 - User → Booking (association)
 - Car → Booking (association)
 - Admin → User (inheritance)
 - Booking → Payment (composition)

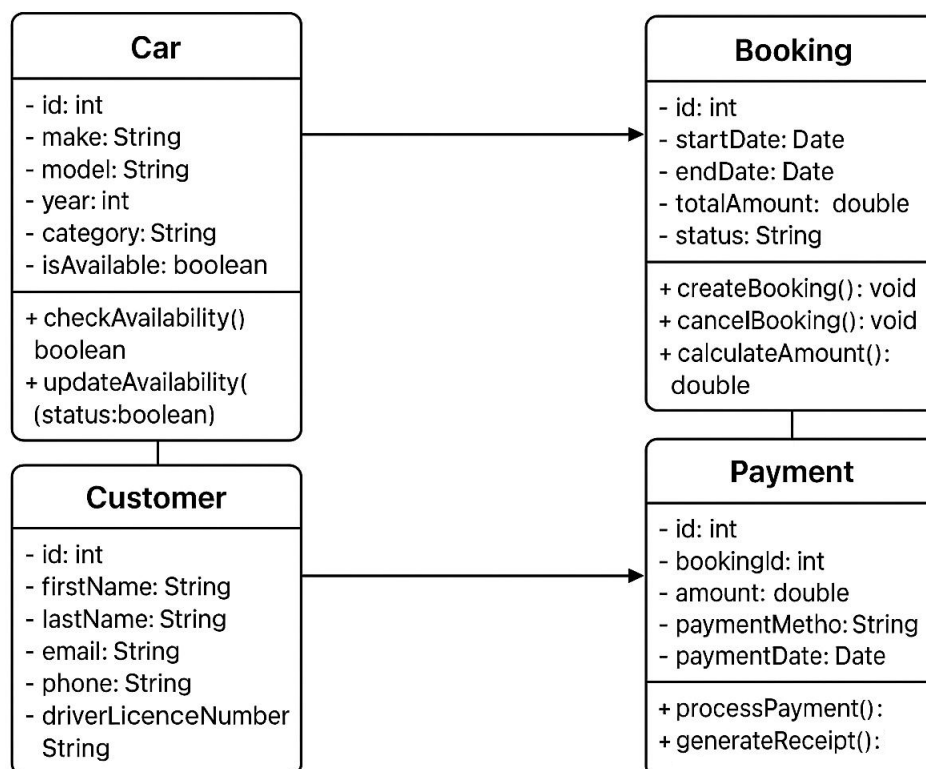


Figure 4.4 Class Diagrams

4.5 SEQUENCE DIAGRAM

The Sequence Diagram shows step-by-step interactions when a customer books a car

- Customer selects car → system checks availability → booking confirmed → payment processed → confirmation sent.

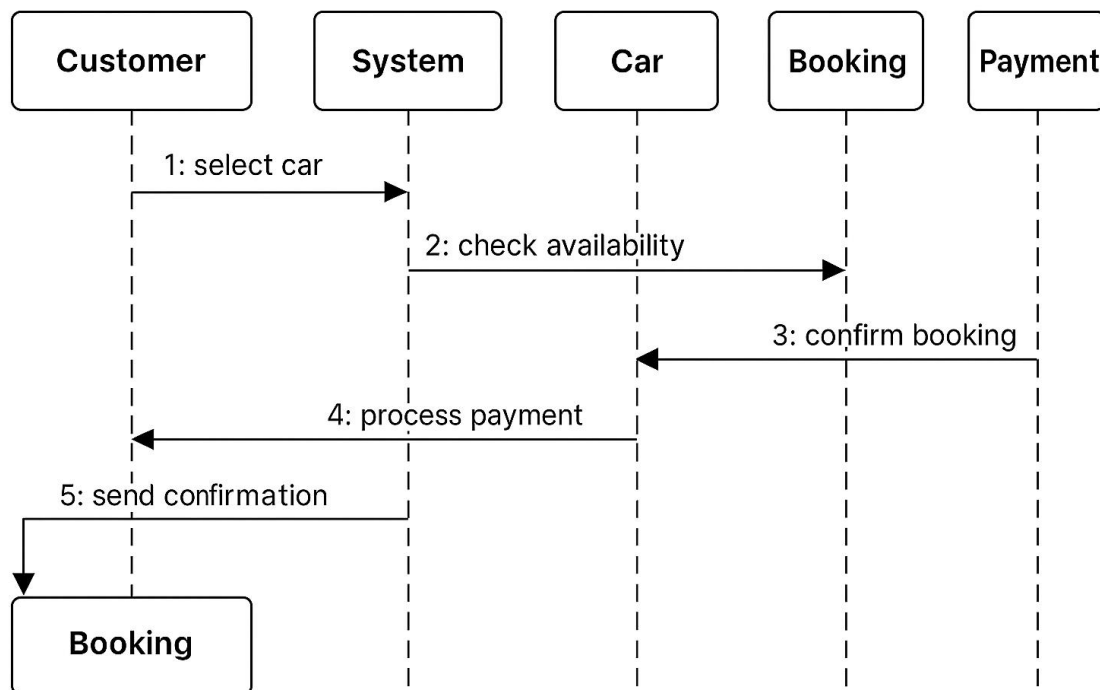


Figure 4.5 Sequence Diagram

4.6 ACTIVITY DIAGRAM

An Activity Diagram is a UML behavioral diagram that represents the sequence of activities and the flow of control in a system. It is widely used to model workflows and user interactions in order to capture both sequential and parallel actions within the application.

In the Self-Drive Car Booking System, the activity diagram visually illustrates how different users (car owners, renters, and administrators) and the system interact to complete key processes. It includes activities such as user registration and authentication, car listing and management, browsing and searching available vehicles, making a booking request, confirming payment, and tracking booking status.

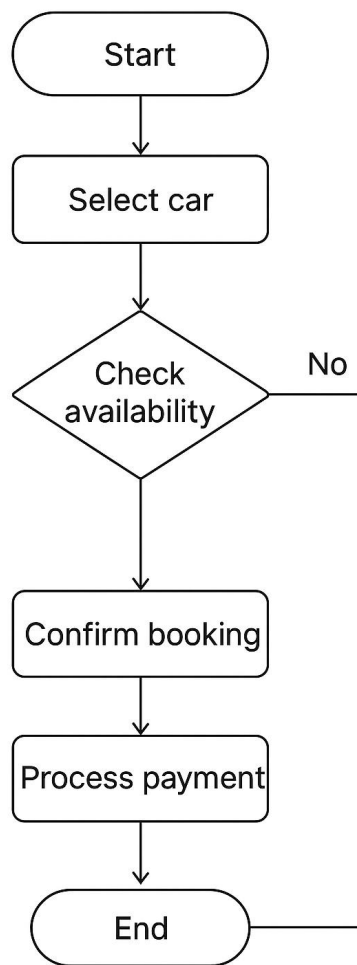


Figure 4.6 Activity Diagram

4.3.2 DATA STRUCTURES

Data Structures are the way you organize, store, and manage data so that it can be processed efficiently. They provide a systematic format to store data in memory or a database and allow easy insertion, retrieval, updating, and deletion of information.

Table 4.1 User Table

S.no	Field Name	Null	Type
1	User_Id	Not Null	Int
2	Name	Not Null	Varchar
3	Email	Not Null	Varchar
4	Password	Not Null	Varchar
5	Role	Not Null	Varchar

Table 4.2 Car Table

S.no	Field Name	Null	Type
1	Car_id	Not Null	Int
2	Owner_id	Not Null	Int
3	Brand	Not Null	Varchar
4	Model	Not Null	Varchar
5	Year	Not Null	Int
6	Registration_no	Not Null	Varchar

7	Price_per_day	Not Null	Decimal
8	Status	Not Null	Enum

Table 4.3 Booking Table

S.no	Field Name	Null	Type
1	Booking_id	Not Null	Int
2	Car_id	Not Null	Int
3	User_id	Not Null	Int
4	Start_date	Not Null	Date
5	End_date	Not Null	Date
6	Total_amout	Not Null	Decimal
7	Status	Not Null	Enum

4.4 USER INTERFACE DESIGN

User Interface Design in a project is the process of designing the screens, pages, and visual elements (like buttons, menus, input boxes, forms) that allow users to interact with the system easily and effectively.

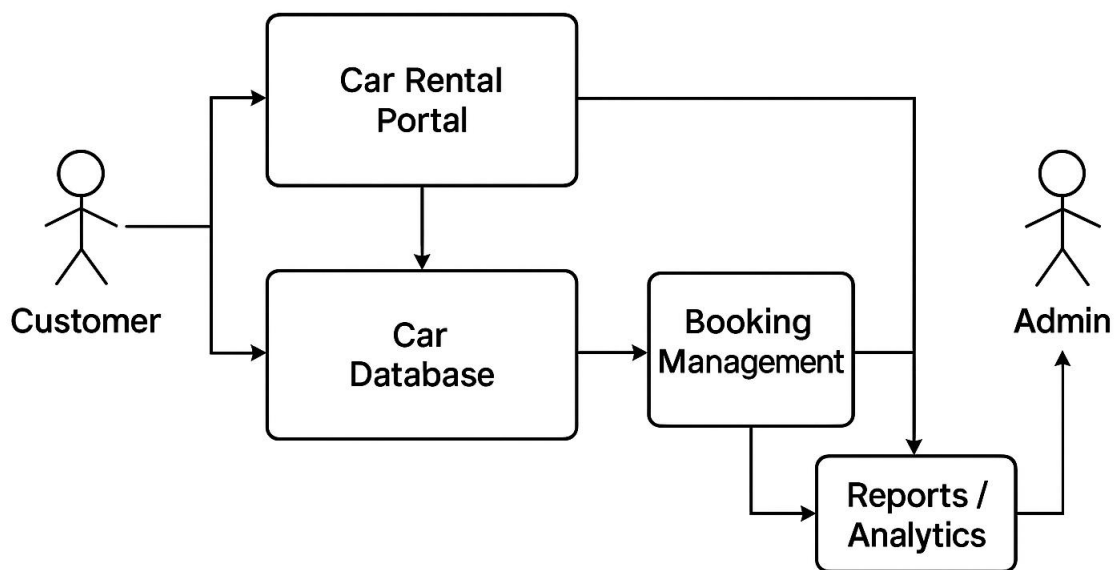


Figure 4.7 User Interface Design

4.5 RISK LIST

Every project carries certain risks that can affect its success, performance, and long-term sustainability. For a Self-Drive Car Booking System, the risks primarily involve data security, payment reliability, vehicle availability, user trust, and system performance. Issues such as unauthorized access, fraudulent bookings, or technical failures can disrupt operations and reduce user confidence. Identifying these risks early and implementing effective mitigation strategies ensures that the platform remains secure, reliable, and user-friendly. Proper handling of these risks also helps maintain trust between car owners and renters while ensuring smooth booking and payment processes.

Table 4.4 Risk List

Risk Ranking magnitude	Risk Description and Impact	Mitigation Strategy
1	Unauthorized access to user/car/booking data could lead to privacy breaches and loss of trust	Strong authentication, encryption, and role-based access control
2	Payment gateway failures may block or delay bookings.	Integrate multiple gateways, retry mechanisms, and real-time monitoring.
3	Server downtime may interrupt bookings and payments.	Use cloud hosting, auto-scaling, backups, and disaster recovery planning.

4.6 TEST CASE DESIGN

Test Case Design is the process of preparing test cases that validate whether the project functions correctly under different scenarios. It specifies the test ID, module, scenario, steps, expected result, and status.

Table 4.5 Test Case Design

Test Case ID	Module	Test Scenario	Test Steps	Expected Results	Status (pass/fail)
TC01	User Registration	Register a new user	Open registration form → Enter valid details → Submit	User account created successfully, confirmation sent	Pass
TC02	User Login	Authenticate user credentials	Enter correct and incorrect credentials → Submit	Correct → Login successful; Incorrect → Access denied	Pass
TC03	Car Listing	Add new car	Owner login → Add car details → Submit	Car listed successfully	Pass
TC04	Car Booking	Book a car	Select car → Enter dates → Confirm booking	Booking confirmed and saved	Pass
TC05	Payment Processing	Process rental payment	Choose booking → Pay via gateway → Submit	Payment successful and booking status updated	Pass
TC06	Payment Failure Handling	Simulate failed transaction	Cancel/Fail payment	System shows "Payment Failed", no booking recorded	Pass

TC07	Unauthorized Access	Access admin features as normal user	Login as customer → Try admin panel	Access denied with warning message	Pass
TC08	Report Generation	Generate report	Login as admin → Select "Reports" → Generate	Report generated with accurate data	Pass
TC09	Data Backup & Recovery	Test system during database crash	Disconnect database → Perform donation → Reconnect	System handles gracefully, no data loss	Pass
TC10	Logout Functionality	User logs out of system	Click on logout button	User session terminated, redirected to login page	Pass